Centro Federal de Educação Tecnológica de Minas Gerais
Programa de Pós-Graduação em
Modelagem Matemática e Computacional

Algoritmos e Estruturas de Dados

TRABALHO PRÁTICO: Implementações para o Problema do Caixeiro Viajante (PCV)

ALUNO: BRASILIO ALVES FREITAS
MATRÍCULA: 20235002202

06 de julho de 2023

Sumário

1. Problema do Caixeiro Viajante 3
2. Métodos heurísticos aplicados ao problema 4
    2.1. Algoritmo do Vizinho mais Próximo 5
    2.2. Método da Descida Completa 5
    2.3. Método da Descida Randômica 5
    2.4. Iterated Local Search 6
3. Parte 1 – Força Bruta 6
    3.1. Geração das instâncias 6
    3.2. Algoritmo - Força Bruta 7
    3.3. Resultados Computacionais 8
4. Parte 2 – Métodos heurísticos aplicados ao PCV 9
    4.1. Instâncias testadas 9
    4.2. Métodos Heurísticos (Algoritmos) 10
    4.3. Resultados Computacionais 13
5. Conclusões 13

1- Problema do Caixeiro Viajante (*Travelling Salesman Problem*)

O problema do caixeiro viajante é um problema clássico de otimização combinatória. A primeira coisa que podemos pensar, para resolver esse tipo de problema, é reduzi-lo a um problema de enumeração: achamos todas as rotas possíveis e, manualmente ou usando um computador, calculamos o comprimento de cada uma delas e, então, teremos a menor. Percebe-se que o problema é de enumeração pelo fato de contarmos todas elas.

O Problema do Caixeiro Viajante (PCV) é assim descrito: dado um conjunto de cidades, no qual existe um caminho entre duas quaisquer cidades, com distância positiva, determinar o caminho de menor distância que, a partir de uma cidade de origem, percorre todas as cidades e retorna à cidade inicial, de modo que a mesma cidade só seja visitada apenas uma vez.

O PCV pode ser modelado a partir de um grafo não direcionado e, computacionalmente, por meio de matrizes simétricas ou triangulares, sendo a diagonal principal destas matrizes nulas.

A Figura 1 apresenta um exemplo de um PCV para $n = 4$ cidades:

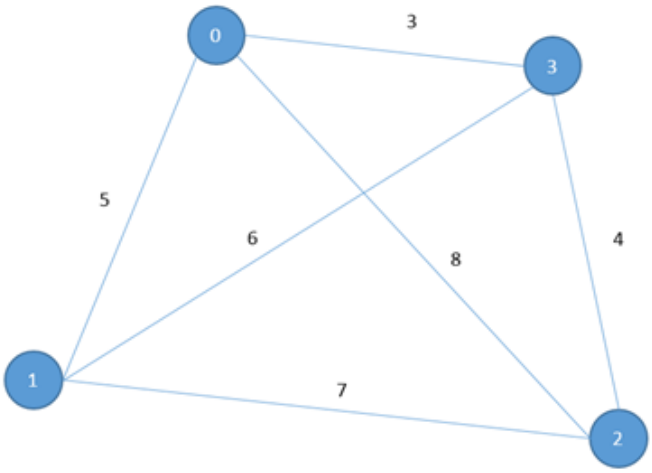

Figura 1: PCV para n=4

A partir da Figura 1, o grafo pode ser representado pela seguinte matriz de adjacências:

$$\begin{pmatrix} 0 & 5 & 8 & 3 \\ 5 & 0 & 7 & 6 \\ 8 & 7 & 0 & 4 \\ 3 & 6 & 4 & 0 \end{pmatrix}$$

Uma das formas de se resolver o problema é utilizando um algoritmo de força bruta, o qual consiste em testar todos os caminhos possíveis e encontrar a solução de menor distância. No entanto, para n cidades, o algoritmo de força bruta deverá testar exatamente $(n-1)!$ possíveis caminhos, o que torna o processo inviável quando n cresce muito. Para problemas menores, como o apresentado pela Figura 1, temos $(4-1)! = 3! = 6$ possíveis caminhos diferentes e um algoritmo de força bruta pode determinar a solução rapidamente. A Figura 2 apresenta todos os caminhos possíveis para o exemplo, bem como as distâncias, para o exemplo apresentado.

```
0  - Solucao testada: x=   [0, [1, 2, 3], 0] f(x)=  19
1  - Solucao testada: x=   [0, [1, 3, 2], 0] f(x)=  23
2  - Solucao testada: x=   [0, [2, 1, 3], 0] f(x)=  24
3  - Solucao testada: x=   [0, [2, 3, 1], 0] f(x)=  23
4  - Solucao testada: x=   [0, [3, 1, 2], 0] f(x)=  24
5  - Solucao testada: x=   [0, [3, 2, 1], 0] f(x)=  19
```

Figura 2: Possíveis soluções do PCV apresentado

Para o PCV apresentado, temos a seguinte solução ótima: 0 – 1 – 2 – 3 – 0

O que resulta na distância mínima igual a 19. A Figura 3 apresenta o resultado do trajeto de menor caminho.

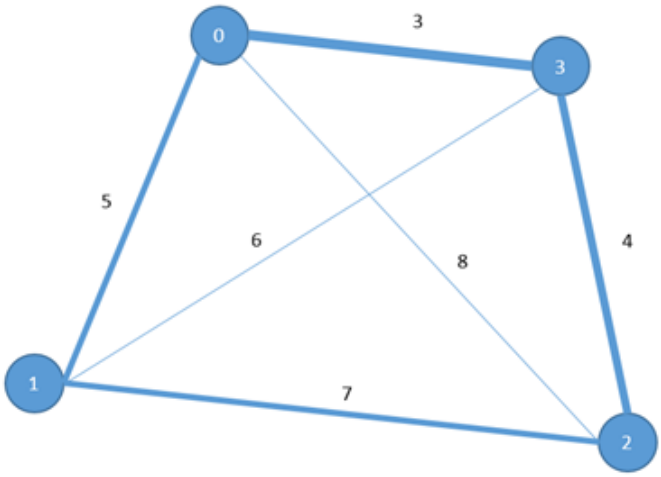

Figura 3: Solução do exemplo do PCV

Mas como proceder para problemas maiores? O tempo computacional cresce muito rápido à medida que os valores de n aumentam. Para valores de n entre 12 e 15, algoritmos de força bruta já começam a ficar demasiadamente lentos e, a partir de $n > 15$, pode não ser possível resolver por este método.

Então, como resolver problemas com quantidades cada vez maiores de cidades? Para estes casos, são recomendados algoritmos heurísticos, os quais não garantem a solução ótima, mas podem retornar soluções aceitáveis, dentro de tempos computacionais também aceitáveis, mas sem garantir a otimalidade da solução encontrada. Neste trabalho foram implementados métodos para resolver problemas com $n = 561$, $n = 535$ e $n = 1032$ cidades.

Para tanto foram usadas as seguintes heurísticas:

- Heurística do Vizinho Mais Próximo;
- Método da Descida Completa;
- Método da Descida Randômica;
- *Iterated Local Search* – ILS.

Para instâncias menores, geradas de forma aleatória, foi aplicado o Algoritmo de Força Bruta, e foi comprovado, por meio de testes computacionais, que seu crescimento de tempo computacional é de ordem exponencial, justificando assim a utilização de métodos heurísticos aos problemas maiores. A próxima seção será destinada a uma breve explicação dos métodos heurísticos utilizados neste trabalho.

2 - Métodos heurísticos aplicados ao problema

Os algoritmos heurísticos não garantem que a solução ótima do problema será encontrada, no entanto, garante soluções factíveis dentro de tempos computacionais satisfatórios, de acordo com o problema tratado.

O algoritmo do vizinho mais próximo é uma heurística gulosa, que retorna uma solução do problema. A partir desta solução, foi aplicado o método ILS para buscar melhorar a solução obtida. Como estratégia de busca local aplicado ao algoritmo ILS, foram testadas as heurísticas Descida Completa e Descida Randômica.

2.1 - Algoritmo do Vizinho mais Próximo

O algoritmo o vizinho mais próximo consiste em:

Passo 1: Iniciar em vértice arbitrário;

Passo 2: Buscar o vértice mais próximo ainda não visitado, com base nos pesos das arestas e mover até este vértice;

Passo 3: Repetir o Passo 2 até que o último vértice seja visitado;

Passo 4: Retornar ao vértice inicial (Passo 1).

Esta heurística foi utilizada para construir uma solução inicial para o problema, a qual foi utilizada no algoritmo ILS, com o objetivo de refinamento.

2.2 - Método da Descida Completa

O método da descida consiste em, dada uma solução inicial para o problema, avaliar toda a vizinhança desta solução, de acordo com um movimento definido e, caso uma solução que melhore o valor da função objetivo seja encontrada, trocar a solução do problema. O processo se repete até que um determinado critério de parada seja atingido, em geral usa-se como critério de parada o número máximo de iterações sem melhora.

O algoritmo da Descida utilizado neste trabalho consiste em:

Passo 1: Iniciar com uma solução inicial S;

Passo 2: Gerar todos os vizinhos de S, denotados por S';

Passo 3: Caso algum vizinho S' seja melhor do que S, trocar S por S', e repetir o passo 2 para a nova solução S;

Passo 4: O processo se repete até que todos os vizinhos sejam testados.

No caso, um vizinho é gerado pelo movimento de trocar duas posições contíguas de S.

No algoritmo da Descida, todos os vizinhos são testados.

2.3 - Método da Descida Randômica

O método da descida randômica busca evitar a busca exaustiva de toda a vizinhança de uma solução. Neste trabalho, o algoritmo da descida randômica aplicado foi:

Passo 1: Iniciar com uma solução inicial S;

Passo 2: Gerar um vizinho de S, denotado por S', por meio de um movimento definido.

Passo 3: caso S' seja melhor do que S, troca S por S', caso contrário, incrementar *iter*;

Passo 4: O processo se repete até *iter* atingir um valor máximo sem melhora, denotado por *itermax*.

A Descida Randômica implementada neste trabalho considera, como vizinho de S, uma solução S' gerada pela troca de duas posições (que não seja a origem e o destino), escolhidas de forma aleatória, com probabilidade uniforme. Este conceito de vizinho foi utilizado para deixar o método com maiores características aleatórias.

2.4 - *Iterated Local Search*

O algoritmo *Iterated Local Search* (ILS) consiste em uma metaheurística, pois possui estratégia para tentar fugir de ótimos locais. Para isso, o ILS faz perturbações na solução corrente e, em seguida, aplica algum algoritmo de busca local.

O ILS aplicado neste trabalho consiste em:

Passo 1: Iniciar com uma solução inicial S;

Passo 2: Aplicar busca local (Descida ou Descida Randômica);

Passo 3: Efetuar uma perturbação na solução retornada pela Busca Local, gerando uma nova solução e aplicar novamente a busca local;

Passo 4: Caso a solução retornada pela busca local melhore a solução corrente, trocar a solução e recomeçar o processo de perturbação, caso contrário, incrementar o nível de perturbação até que o critério de parada seja atingido.

A perturbação da solução utilizada neste trabalho consiste em:

Perturbação de nível 1: trocar duas posições escolhidas de forma aleatória, com probabilidade uniforme;

Perturbação de nível 2: trocar dois pares de posições, escolhidas de forma aleatória, com probabilidade uniforme;

Perturbação de nível 3: trocar três pares de posições, escolhidas de forma aleatória, com probabilidade uniforme;

Perturbação de nível n: trocar n pares de posições, escolhidas de forma aleatória, com probabilidade uniforme.

Uma caraterística das perturbações é que não podem ser muito pequenas, pois assim não conseguem fugir dos ótimos locais e não podem ser muito grandes, para não gerar outra solução de forma totalmente aleatória.

3- Parte 1: Algoritmo da Força Bruta

A Parte 1 deste trabalho consiste em aplicar o algoritmo da Força Bruta para resolver instâncias geradas de forma aleatória. Para implementação do algoritmo foi escrito um código em Phyton. O código é dividido em três partes: geração de instâncias de forma aleatória, solução das instâncias e permutação das possíveis soluções.

A solução e o tempo computacional foram coletados e os resultados estão representados através da Tabela 1.

3.1 - Geração das instâncias

As instâncias foram geradas por meio de matrizes de adjacências, simétricas, com diagonal principal nula e com pesos inteiros, gerados de forma aleatória, entre 1 e 10. A Figura 4 apresenta o algoritmo utilizado para a geração das instâncias:

```python
def retornaMatrizSimestricaAleatoria(tamanho):
    #tamanho=100;

    m=np.zeros( (tamanho, tamanho) )
    for i in range(tamanho):
        for j in range(tamanho):
            if j!=i:
                m[i][j] = random.randint(1,10)

            if m[i][j]!=m[j][i]:
                m[j][i]=m[i][j]

    return m
```

Figura 4: Função utilizada para geração das instâncias aleatórias

3.2 - Algoritmo - Força Bruta

O algoritmo da Força Bruta, Figura 5, que foi aplicado à solução das instâncias geradas.

```python
import numpy as np
import time
import timeit
import random
from datetime import datetime

#bloco de definição de variáveis e funções


def fb(grafo, s):
    vertex = []
    vertex2 = []
    for i in range(v): #Gera vetor dos vértices
        if i != s:
            vertex.append(i)

    custo_min = 999999999
    iter = 0
    vertex2=vertex.copy()
    solucao = [0,vertex,0]
    while True:

        custo_atual = 0
        k = s
        for i in range(len(vertex)):
            custo_atual += grafo[k][vertex[i]] #Atualiza custo da rota da permutação analisada
            k = vertex[i]
        custo_atual += grafo[k][s]
        if custo_atual < custo_min:
            vertex2=vertex.copy()
            solucao=np.array([0,vertex2,0])
        custo_min = min(custo_min, custo_atual) #Caso o custo atual seja menor do que o da permutação de custo mínimo, trocar
        print (iter, ' - Solucao testada: x=  ', [0, vertex, 0],"f(x)= ", custo_atual) #imprimir a solução atual na tela

        iter = iter + 1
        if not permutacao(vertex): #Gerar nova permutação para avaliar a solução
            break
    return [custo_min,solucao, iter]
```

Figura 5: Algoritmo – Força Bruta

As soluções possíveis foram geradas pela função de permutação, Figura 6.

```python
def permutacao(l):
    n = len(l)
    i = n - 2

    while i >= 0 and l[i] > l[i + 1]:
        i -= 1

    if i == -1:
        return False

    j = i + 1
    while j < n and l[j] > l[i]:
        j += 1

    j -= 1

    #Permutando posições e gerando as novas soluções para analisar

    l[i], l[j] = l[j], l[i]
    left = i + 1
    right = n - 1

    while left < right:
        l[left], l[right] = l[right], l[left]
        left += 1
        right -= 1
    return True
```

Figura 6: Função de permutação utilizada no algoritmo da Força Bruta

3.3 – Resultados Computacionais

A Tabela 1 mostra os resultados computacionais obtidos após as soluções das instâncias.

n (cidades)	Distância	Tempo (s)
2	16	0,000302
3	13	0,000487
4	17	0,000503
5	21	0,000548
6	29	0,001632
7	26	0,015798
8	22	0,116402
9	25	0,610903
10	23	5,806156
11	17	62,033303
12	20	825,10419

Tabela 1: Resultados computacionais – Força Bruta

A Tabela 1 apresenta os resultados computacionais obtidos pelo método da Força Bruta, para diferentes números de cidades (valores de n). Devido ao grande crescimento do tempo computacional, o método foi executado para um valor máximo de $n = 12$.
Com os tempos computacionais e o número de cidades, utilizando o *Software QtiPlot*, foi construído um gráfico de dispersão e realizado uma regressão não linear, com posterior inserção de uma curva de tendência (gráfico 1 e no script de saída do *QtiPlot*).
Pelo gráfico percebe-se o comportamento exponencial do crescimento do tempo computacional, o que já elevaria "muito" o tempo para execução de uma instância contendo 13 cidades.

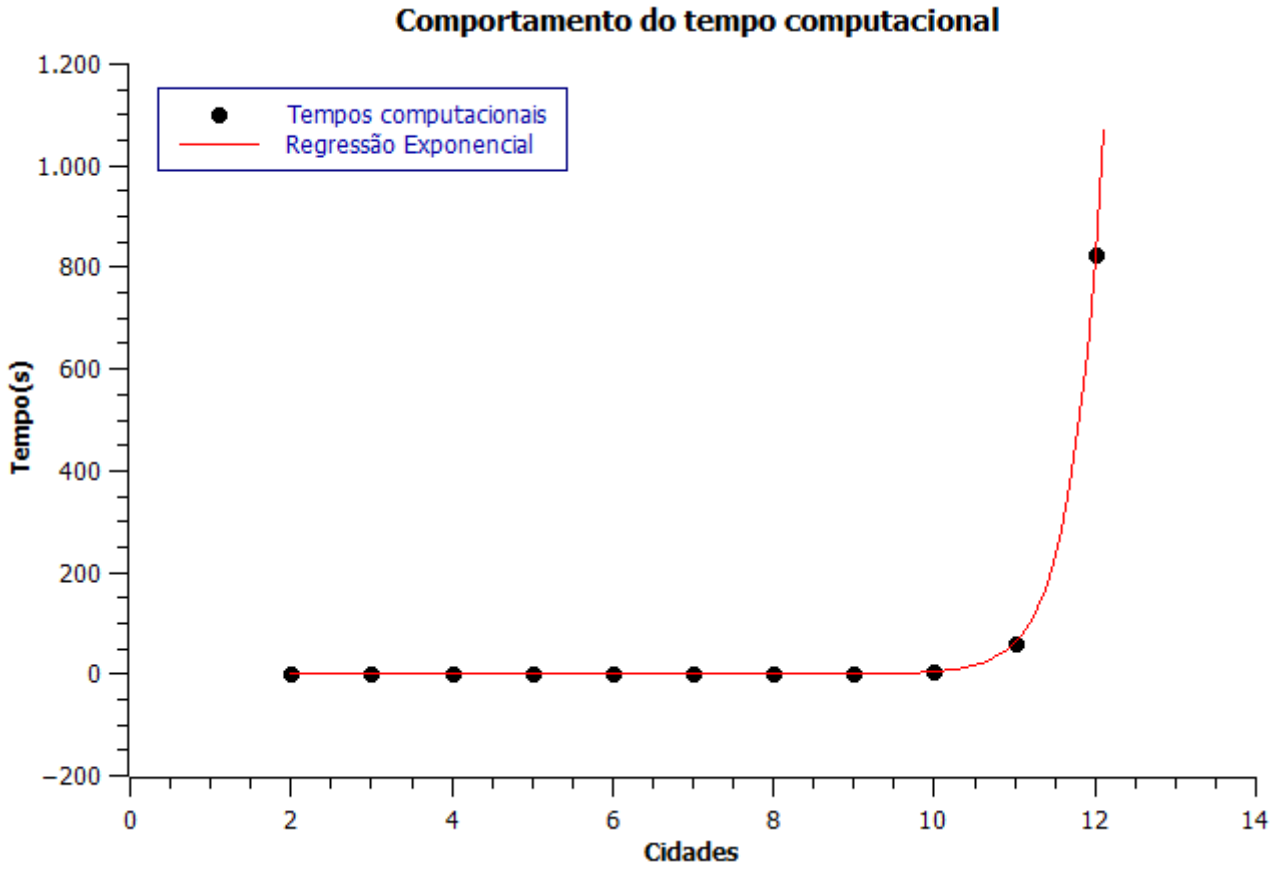

Gráfico 1 – Tempo computacional x número de cidades

Script de saída do *Software* **Qtiplot**:

```
[06/07/2023 14:25:33 Plot: "Graph1"]
Exponential growth of dataset: Table1_2, using function: y0+A*exp(x/t)
Weighting Method: No weighting
Scaled Levenberg-Marquardt algorithm with tolerance = 0,0001
From x = 2,000000000000000e+00 to x = 1,200000000000000e+01
A (amplitude) = 2,696082629291231e-11 +/- 2,089717207266467e-12
t (lifetime) = 3,864490946260836e-01 +/- 9,643429854391659e-04
y0 (offset) = 1,515526918745977e-01 +/- 1,288010600245068e-01
--------------------------------------------------------------------------------
Chi^2/doF = 1,439117991407264e-01
R^2 = -0,118183616408917
Adjusted R^2 = -0,597405166298454
RMSE (Root Mean Squared Error) = 292,491137434382
RSS (Residual Sum of Squares) = 684.408,523821269
--------------------------------------------------------------------------------
Iterations = 183
Status = success
--------------------------------------------------------------------------------
```

O crescimento do tempo de processamento para solução do problema se dá devido ao elevado número de soluções possíveis, sendo $(n-1)!$ soluções possíveis para um conjunto de n cidades. Deste modo, para valores maiores do que o testado, a utilização da força bruta demandará elevado tempo computacional, justificando a utilização de Métodos Heurísticos, os quais não garantem a solução ótima, mas garante uma solução factível, em menor tempo computacional.

O crescimento do número de cidades eleva muito a quantidade de soluções distintas a serem testadas. Para a execução limite que foi feita, $n = 12$, foram testadas $(12-1)! = 11! = 39.916.800$ soluções, ao aumentar para $n = 13$ cidades, seriam $479.001.600$ soluções distintas a serem testadas, o que representaria um enorme crescimento no tempo de processamento.

4 - Parte 2: Métodos Heurísticos aplicados ao PCV

A primeira heurística implementada foi a do Vizinho mais Próximo, com a qual foi gerada uma solução inicial, utilizada como solução inicial da Metaheurística ILS (*Iterated Local Search*). O ILS foi testado com duas heurísticas de busca local distintas, a Descida Completa e a Descida Randômica.

4.1 - Instâncias testadas

Foram testadas três instâncias, pa561.tsp, si535.tsp e si1032.tsp, as quais contém, respectivamente, 561, 535 e 1032 cidades. Para ler as instâncias e transformar o conteúdo dos arquivos em matrizes de adjacências, foi implementado o algoritmo conforme Figura 7.

```python
def ler_dados(filename):
    if filename == 'si535.tsp' or filename == 'si1032.tsp':
        f = open(filename) #abre o arquivo
        line = f.readline()
        while line.find("DIMENSION") == -1: #procura a dimensão para salvar a qtde de cidades
            line = f.readline()
        n = int(line.split()[-1])
        c = np.zeros([n,n])
        i,j = 0,0

        while line.find("EDGE_WEIGHT_SECTION") == -1: #começa a ler os pesos após os textos iniciais
            line = f.readline()
        data = '0'
        while data != "EOF": #Critério para parar de ler os números das linhas
            line=f.readline() #ler a linha
            for data in line.split(): #faz um split do conteúdo e lê cada número
                if data != 'EOF':
                    c[i,j]=int(data) #atribuições para formar matriz simétrica de pesos
                    c[j,i]=int(data)  #atribuições para formar matriz simétrica de pesos
                j+= 1 #incrementa próxima coluna da matriz de pesos
                if j>=n: #incrementa próxima linhada matriz de pesos
                    i+= 1
                    j=i
    elif filename == 'pa561.tsp':
        f = open(filename) #abre o arquivo
        line = f.readline()
        while line.find("DIMENSION")==-1: #procura a dimensão para salvar a qtde de cidades
            line = f.readline()
        n=int(line.split()[-1])
        c=np.zeros([n,n])
        i=0

        while line.find("EDGE_WEIGHT_SECTION")==-1: #começa a ler os pesos após os textos iniciais
            line=f.readline()
        data='0'
        while data!='DISPLAY_DATA_SECTION': #Critério para parar de ler os números das linhas
            j=0
            line = f.readline() #ler a linha
            for data in line.split(): #faz um split do conteúdo da linha e lê cada número
                if data!='DISPLAY_DATA_SECTION':
                    c[i,j]=int(data)
                    c[j,i]=int(data)
                    j+=1
            i+=1
    else: return('Instância desconhecida')

    return c
```

Figura 7: Função para ler as instâncias e criar as matrizes de adjacências

4.2 - Algoritmos dos Métodos Heurísticos

Nesta Seção, serão apresentados os algoritmos implementados. A Heurística gulosa do Vizinho mais Próximo é apresentada pela Figura 8.

```python
def vizproximo(grafo,s):
    print("-------------- Construção Gulosa pelo Vizinho mais Próximo ---------------------")
    print('Vértice inicial: ',s)
    ultimo = s
    n=grafo.shape[0]
    vertex=[]
    custo = 0
    for i in range(n): #Gera vetor dos vértices
        if i != s:
            vertex.append(i)

    visits = []
    visits.append(s) #marca o primeiro vértice como visitado

    for i in range(n-1):
        aux = proxvertice(grafo,s, visits) #indice e distancia do vértice mais próximo de s (ainda não visitado)
        s=aux[0]
        custo = custo+aux[1]
        visits.append(s) #marca o vértice escolhido como visitado
        print ('Próximo vértice: ', s, 'Custo atual: ', custo)

    custo = custo+grafo[s,ultimo] #soma o custo para voltar ao inicio
    visits.append(ultimo)

    print('Último vértice: ', ultimo,'Custo total: ', custo)

    return visits
```

Figura 8: Heurística do Vizinho mais Próximo

Pode ser observado que a função apresentada pela Figura 8 utiliza, internamente, uma outra função, a que busca qual o vértice mais próximo, de acordo com os pesos das arestas. Essa função, denominada $proxvertice$ é apresentada pela Figura 9.

```
def proxvertice(grafo,s, visits): #funcao para buscar a aresta de menor caminho, a partir de um vertice selecionado s
    indice = 0
    menor = 99999999
    n=grafo.shape[0]

    for i in range(n):
        if i!= s and i not in visits: #exclui da verificação o vértice atual e os vértices já visitados
            if grafo[i,s]<menor:
                menor = grafo[i,s]
                indice = i
    return indice, menor
```

Figura 9: Função que busca o vértice mais próximo

Antes das heurísticas serem apresentadas, será apresentada a função $avaliarsolucao$, utilizada para calcular o valor da função objetivo (distância), correspondente à soma dos pesos das arestas de uma solução. Essa função, apresentada pela Figura 10, foi utilizada em todas as heurísticas implementadas.

```
def avaliarsolucao(grafo, solucao): #função para calcular o custo de uma rota para o PCV
    n=grafo.shape[0]
    f=0
    for i in range (n):
        f=f+grafo[solucao[i],solucao[i+1]]
    return f
```

Figura 10: Função utilizada para calcular o valor da função objetivo (distância)

O algoritmo da Heurística da Descida Completa, Descida Randômica e da Metaheurística ILS são apresentadas pelas Figuras 11, 12 e 13, respectivamente.

```
def descida(grafo, s_ini): #Descida Completa - vizinho: troca contígua
    #são trocadas as posições contíguas
    s=s_ini
    n=grafo.shape[0]
    #iter = 0
    #print('Solução inicial: ', s,' Distância: ', avaliarsolucao(grafo, s))
    #print('Distância Inicial: ', avaliarsolucao(grafo, s))
    i=1
    while i < n-2:
        j=i+1
        #print('i= ', i, 'j= ', j)
        slinha= s.copy()
        aux = slinha[i]
        slinha[i]=slinha[j]
        slinha[j]=aux
        dist1= avaliarsolucao(grafo,s)
        dist2= avaliarsolucao(grafo,slinha)
        #print(dist2)
        if dist2 < dist1: #se a troca melhorar a função objetivo, trocar
            s=slinha.copy()
            dist1=dist2
            print('Aplicando Busca Local (Descida) - Solução atualizada: - Nova Distância: ', dist1, 'Posições trocadas: ', i,'
            i=1
            #print('Solução atualizada: ', s, 'Distância: ', dist2, 'Vértices trocados: ', s[i],' e ', s[j])

        else:
            slinha=s.copy()
            i+=1
    return s
```

Figura 11: Algoritmo da Descida Completa

```python
def descidarand(grafo, s_ini, itermax): #Descida Randômica, o vizinho é definido pela troca de um par, selecionado aleatoriamente
    s=s_ini
    n=grafo.shape[0]
    iter = 0
    #print('Solução inicial: ', s,' Distância: ', avaliarsolucao(grafo, s))
    #print('Distância Inicial: ', avaliarsolucao(grafo, s))
    while iter < itermax:
        #print(iter)
        #random.seed(datetime.now())
        i=random.randint(1,n-1) #não considera o primeiro e o ultimo
        #random.seed(datetime.now())
        j=random.randint(1,n-1)
        while j==i: #não selecionar mesma posição
            #random.seed(datetime.now())
            j=random.randint(1,n-1)
        #print('i= ', i, 'j= ', j)
        slinha= s.copy()
        aux = slinha[i]
        slinha[i]=slinha[j]
        slinha[j]=aux
        dist1= avaliarsolucao(grafo,s)
        dist2= avaliarsolucao(grafo,slinha)
        #print(dist2)
        if dist2 < dist1: #se a troca melhor a função objetivo, trocar
            s=slinha.copy()
            iter = 0
            print('Aplicando Busca Local (Descida Randômica) - Solução atualizada (distância): ', dist2, 'Vértices trocados: ', s
            dist1=dist2
            #print('Solução atualizada: - Nova Distância: ', dist2)
        else:
            slinha=s.copy()
            iter +=1
    return s
```

Figura 12: Algoritmo da Descida Randômica

```python
def ils(grafo,s_ini, itermax, pert): #Metaherística Iterated Local Seach - itermax (Critério da Busca Local), pert (número máx. d

        n=grafo.shape[0]
        s=0

        #Busca Local
        #x=descida(grafo, s_ini)
        x=descidarand(grafo,s_ini, itermax)
        dist1=avaliarsolucao(grafo,x)

        k=1
        while k <= pert:
        #for k in range(pert):
            #criar vetor de trocas
            print("Perturbação: ", k, "pares trocados")
            trocas=[] #vetor de trocas inicial nulo
            m=1
            while m <= 2*k: #montar vetor com os indices para efetuar as trocas de acordo com o nível da perturbação
                #random.seed(datetime.now())
                trocas.append(random.randint(1,n-1))
                #print('trocas: ', trocas, 'm: ', m)
                while m>1 and trocas[m-1] in trocas[0:m-2]: #retirar vértices já escolhidos
                    #random.seed(datetime.now())
                    trocas[m-1]=random.randint(1,n-1)
                m=m+1
            x2=x.copy()
            print('Trocas: ', trocas)
            for i in range(m-2): #realização das trocas (perturbação)
                #print('i: ', i, trocas[i])
                aux=x2[trocas[i]]
                x2[trocas[i]]=x2[trocas[i+1]]
                x2[trocas[i+1]]=aux
            #xlinha= descida(grafo, x2) #Fazer a busca local na solução perturbada
            xlinha=descidarand(grafo,s_ini, itermax)
            dist2=avaliarsolucao(grafo,xlinha)
            print('Busca Local na Solução Perturbada: ', dist2,'- Ótimo até o momento: ', dist1)
            if dist2<dist1:
                x=xlinha.copy()
                dist1=dist2
                print('Solução atualizada: - Nova Distância após perturbação', dist1)
                k=1
            else:
                k+=1

        print('Solução Final pelo ILS: ', dist1)

        return dist1
```

Figura 13: Algoritmo da Metaheurística ILS

4.3 - Resultados Computacionais

Os resultados computacionais são apresentados pela Tabela 2. Para a Descida Randômica foi utilizado, para definição do critério de parada, *itermax=1000*, para o ILS, *n=20* níveis de perturbação.

Instância	f^*	VP1	ILS1	TEMPO 1(s)	GAP1	VP2	ILS2	TEMPO 2(s)	GAP2
pa561.tsp	2763	3422	3378	160,76	22,26%	3422	3418	30,10	23,71%
si535.tsp	48450	50144	50065	76,81	3,33%	50144	50123	22,79	3,45%
si1032.tsp	92650	94571	94571	115,89	2,07%	94571	94571	33,63	2,07%

Tabela 2: Resultados computacionais – Métodos Heurísticos

Na Tabela 2, a coluna Instância contém o nome da instância testada, VP1 corresponde ao valor da função objetivo calculada sobre a solução inicial, gerada pela Heurística do Vizinho mais Próximo e utilizada no ILS com Descida Completa, ILS1 corresponde ao valor da função objetivo da solução retornada pelo ILS com Descida Completa e GAP1 corresponde à diferença da solução encontrada, em porcentagem, em relação ao valor ótimo encontrado na Literatura. VP2 corresponde ao valor da função objetivo calculada sobre a solução inicial, gerada pela Heurística do Vizinho mais Próximo e utilizada no ILS com Descida Randômica, ILS2 corresponde ao valor da função objetivo da solução retornada pelo ILS com Descida Randômica e GAP2 corresponde à diferença da solução encontrada, em porcentagem, em relação ao valor ótimo encontrado na Literatura. As colunas Tempo 1 e Tempo 2 apresentam os tempos computacionais da execução de VP1+ILS1 e VP2+ILS2, respectivamente. A coluna f^* contém os valores ótimos, encontrados em http://elib.zib.de/pub/mp-testdata/tsp/tsplib/stsp-sol.html.

Como pode ser observado pelos resultados apresentados pela Tabela 2, a Heurística do Vizinho mais Próximo gerou boas soluções iniciais, quando comparadas às soluções ótimas das instâncias, principalmente para as instâncias si535.tsp e si1032.tsp. O ILS melhorou a solução inicial para as instâncias pa561.tsp e si535.tsp, mas não conseguiu melhorar o valor da função objetivo para a instância si1032.tsp e, para esta instância maior, outros parâmetros podem ser testados no algoritmo.

5 - Conclusões

Neste trabalho foi possível comprovar, de modo empírico, que o tempo computacional para resolução do Problema do Caixeiro Viajante por meio de algoritmo força bruta cresce de forma exponencial e isso se justifica pela quantidade de soluções a serem testadas, sendo $(n-1)!$ soluções distintas para um problema com n cidades. Para valores de n "grandes" os métodos heurísticos se apresentam como uma ótima opção, visto que buscam soluções factíveis, sem garantia de otimalidade, mas dentro de um tempo computacional aceitável.

A Heurística do Vizinho mais Próximo gera boas soluções e, mesmo aplicando a solução gerada por esta heurística como solução inicial da metaheurística ILS, verifica-se que o ILS não melhora a solução de forma significativa.

Pode ser concluído, de acordo com as instâncias testadas, que os métodos heurísticos são necessários para serem resolvidos problemas com valores de n maiores. Por exemplo, no caso de 1032 cidades, foram gastos 116 e 34 segundos, para se resolver o problema com o ILS1 e ILS2, respectivamente, enquanto que o algoritmo da Força Bruta demorou 825 segundos para resolver um problema com somente 12 cidades.